

Circuit Documentation

1. Circuit Summary

This circuit implements a one-way, 8-bit parallel communication link between two Arduino UNO boards:

- **Sender:** Transmits a text message periodically.
- **Receiver:** Reads the parallel data, generates handshaking signals, and forwards the received bytes through its UART interface at 9600 baud.

The communication protocol is Centronics-style and uses:

- Eight data lines
- One active-low strobe signal
- One active-low acknowledge signal
- One active-high busy signal
- A shared ground connection

The Sender transmits the following message every five seconds:

DATA PTZ: BERAT=2.45g HARDNESS=150N

The message includes a newline character at the end.

Important implementation note: The supplied net list contains three identical Sender-GND-to-Receiver-GND entries. Electrically, these represent one shared ground connection; the duplicate entries do not create additional electrical functions.

2. Component List

2.1 Sender Arduino UNO

Function:

Generates and transmits the parallel data stream.

Configured signals:

- D2–D9: 8-bit data output
- D10: Strobe output
- D11: Acknowledge input
- D12: Busy input

The Sender waits for the Receiver to indicate that it is ready, places each character on the data bus, pulses the strobe line, and waits for the acknowledge response.

2.2 Receiver Arduino UNO

Function:

Receives the parallel data stream and transmits the received bytes through its UART interface.

Configured signals:

- A0–A5 and D6–D7: 8-bit data input
- D3: Strobe input
- D8: Acknowledge output
- D4: Busy output

The Receiver detects the Sender's strobe pulse, reads the eight-bit data value, sends the byte through the UART, and then generates an acknowledge pulse.

3. Component Wiring Details

3.1 Sender Arduino UNO

The Sender uses its digital pins D2 through D9 as an 8-bit parallel output bus. The data pins are assigned in ascending bit order, with D2 carrying the least-significant bit.

Sender Pin	Signal	Connected To
D2	Data bit 0	Receiver A0
D3	Data bit 1	Receiver A1
D4	Data bit 2	Receiver A2
D5	Data bit 3	Receiver A3
D6	Data bit 4	Receiver A4
D7	Data bit 5	Receiver A5
D8	Data bit 6	Receiver D6
D9	Data bit 7	Receiver D7
D10	Active-low strobe output	Receiver D3
D11	Acknowledge input	Receiver D8
D12	Busy input	Receiver D4
GND	Ground reference	Receiver GND

The Sender does not have any other listed electrical connections. In particular, its power pins and unused I/O pins are not connected according to the supplied net list.

3.2 Receiver Arduino UNO

The Receiver receives the eight-bit data bus using a combination of analog-input pins and digital pins.

Receiver Pin	Signal	Connected To
A0	Data bit 0 input	Sender D2
A1	Data bit 1 input	Sender D3
A2	Data bit 2 input	Sender D4
A3	Data bit 3 input	Sender D5
A4	Data bit 4 input	Sender D6
A5	Data bit 5 input	Sender D7
D6	Data bit 6 input	Sender D8

Receiver Pin	Signal	Connected To
D7	Data bit 7 input	Sender D9
D3	Active-low strobe input	Sender D10
D8	Acknowledge output	Sender D11
D4	Busy output	Sender D12
GND	Ground reference	Sender GND

The Receiver's UART output is configured in software, but no external serial device or UART net is included in the supplied electrical net list.

3.3 Parallel Interface Signal Summary

Signal	Direction	Sender Pin	Receiver Pin	Active State
Data bit 0	Sender to Receiver	D2	A0	Logic level
Data bit 1	Sender to Receiver	D3	A1	Logic level
Data bit 2	Sender to Receiver	D4	A2	Logic level
Data bit 3	Sender to Receiver	D5	A3	Logic level
Data bit 4	Sender to Receiver	D6	A4	Logic level
Data bit 5	Sender to Receiver	D7	A5	Logic level
Data bit 6	Sender to Receiver	D8	D6	Logic level
Data bit 7	Sender to Receiver	D9	D7	Logic level
Strobe	Sender to Receiver	D10	D3	Active low
Acknowledge	Receiver to Sender	D11	D8	Active low pulse
Busy	Receiver to Sender	D12	D4	High when busy
Ground	Common reference	GND	GND	Common

4. Communication Operation

4.1 Sender Operation

For each character:

1. The Sender checks the Busy input.
2. It waits while the Receiver indicates that it is busy.
3. The character is placed on D2–D9.
4. The Sender waits briefly for the data lines to stabilize.
5. The strobe line on D10 is driven low.
6. The strobe remains low for approximately 2 ms.
7. The strobe returns high.
8. The Sender waits for the Receiver to pull the acknowledge line low.
9. The Sender then waits for the acknowledge line to return high.
10. The next character is transmitted.

The Sender transmits the complete text once every five seconds.

4.2 Receiver Operation

For each received character:

1. The Receiver indicates that it is not busy.
2. It waits for the strobe input to become low.
3. It immediately asserts the Busy output.
4. It reads the eight-bit parallel value.
5. It forwards the byte through the UART.
6. It waits for the strobe signal to return high.
7. It generates an active-low acknowledge pulse.
8. It releases the acknowledge line back to high.
9. It repeats the process for the next character.

5. Documented Code

5.1 Sender Firmware

Pin Assignments

```
const int dataPins[8] = {2, 3, 4, 5, 6, 7, 8, 9};  
const int STROBE_PIN = 10;  
const int ACK_PIN = 11;  
const int BUSY_PIN = 12;
```

- dataPins[0] through dataPins[7] form the eight-bit data bus.
- STROBE_PIN initiates a transfer.
- ACK_PIN confirms that the Receiver has read the byte.
- BUSY_PIN indicates whether the Receiver is ready.

Initialization

```
void setup() {  
    for (int i = 0; i < 8; i++) {  
        pinMode(dataPins[i], OUTPUT);  
    }  
  
    pinMode(STROBE_PIN, OUTPUT);  
    pinMode(ACK_PIN, INPUT_PULLUP);  
    pinMode(BUSY_PIN, INPUT_PULLUP);  
  
    digitalWrite(STROBE_PIN, HIGH);  
  
    delay(2000);  
}
```

The data and strobe lines are outputs. The acknowledge and busy lines use internal pull-up resistors and are inputs. The strobe signal is inactive high during standby.

Main Loop

```
void loop() {  
    kirimTeks("DATA PTZ: BERAT=2.45g HARDNESS=150N\n");  
    delay(5000);  
}
```

The Sender transmits the complete message, waits five seconds, and repeats.

Text Transmission

```
void kirimTeks(String teks) {  
    for (int i = 0; i < teks.length(); i++) {  
        kirimByte(teks[i]);  
    }  
}
```

Each character in the string is passed individually to kirimByte().

Byte Transmission

```
void kirimByte(char karakter) {
    while (digitalRead(BUSY_PIN) == HIGH) {
        // Wait while the Receiver is busy
    }

    for (int i = 0; i < 8; i++) {
        digitalWrite(dataPins[i], (karakter >> i) & 0x01);
    }

    delayMicroseconds(2);

    digitalWrite(STROBE_PIN, LOW);
    delay(2);
    digitalWrite(STROBE_PIN, HIGH);

    while (digitalRead(ACK_PIN) == HIGH) {
        // Wait for acknowledge to go low
    }

    while (digitalRead(ACK_PIN) == LOW) {
        // Wait for acknowledge to return high
    }
}
```

The character is transmitted least-significant bit first across the data bus:

- Bit 0 is output on D2.
- Bit 1 is output on D3.
- Continuing through bit 7 on D9.

The Sender waits indefinitely for Busy and Acknowledge transitions. If the Receiver is disconnected or fails to generate the expected signals, the Sender can remain blocked in one of these loops.

5.2 Receiver Firmware

UART Initialization

```
void uart_init(uint32_t baudrate) {
    const uint32_t divisorFP1024 =
```

```

        (((uint32_t)(1024 / 8) * F_CPU) / baudrate);

const uint16_t divisor = (divisorFP1024 + 512) / 1024;
const uint16_t ubrr = divisor - 1;

UBRR0H = ubrr >> 8;
UBRR0L = ubrr & 0xff;

UCSR0A |= (1 << U2X0);
UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
UCSR0B = (1 << RXEN0) | (1 << TXEN0);
}

```

The UART is configured for:

- 9600 baud
- 8 data bits
- Transmit and receive enabled
- Double-speed mode enabled

Only the transmit path is used by the parallel-to-serial application.

UART Byte Transmission

```

void uart_putbyte(uint8_t byte) {
    loop_until_bit_is_set(UCSR0A, UDRE0);
    UDR0 = byte;
}

```

This function waits until the UART transmit buffer is available and then writes the received parallel byte to the UART data register.

Receiver Signal Definitions

```

#define PAR_STROBE_BIT 3
#define PAR_SEL_BIT    1
#define PAR_BUSY_BIT   4
#define PAR_ACK_BIT    0

```

The receiver firmware uses AVR port-bit definitions:

- Strobe: Port D bit 3, corresponding to Arduino D3
- Busy: Port D bit 4, corresponding to Arduino D4
- Acknowledge: Port B bit 0, corresponding to Arduino D8

- Select: Port B bit 1, corresponding to Arduino D9

The Select signal is not present in the supplied wiring and is not connected to the Sender.

Parallel Port Initialization

```
void parport_init(void) {
    DDRB |= (1 << PAR_SEL_BIT);
    PORTB &= ~(1 << PAR_SEL_BIT);

    DDRD |= (1 << PAR_BUSY_BIT);
    PORTD &= ~(1 << PAR_BUSY_BIT);

    PORTD |= (1 << PAR_STROBE_BIT);

    DDRB |= (1 << PAR_ACK_BIT);
    PORTB |= (1 << PAR_ACK_BIT);

    PORTB |= (1 << PAR_SEL_BIT);
}
```

Initialization performs the following actions:

- Configures Select as an output.
- Configures Busy as an output.
- Sets the initial Busy state low.
- Enables the strobe pull-up.
- Configures Acknowledge as an output.
- Sets Acknowledge inactive high.
- Sets Select high to indicate that the interface is available.

Because Select is not included in the circuit wiring, its state does not affect the documented Sender-to-Receiver transfer.

Parallel Data Reception

```
void parport_loop() {
    PORTD &= ~(1 << PAR_BUSY_BIT);

    loop_until_bit_is_clear(PIND, PAR_STROBE_BIT);

    PORTD |= (1 << PAR_BUSY_BIT);

    const uint8_t byte = (PINC & 0x3f) | (PIND & 0xc0);
    uart_putbyte(byte);
}
```

```

loop_until_bit_is_set(PIND, PAR_STROBE_BIT);

PORTB &= ~(1 << PAR_ACK_BIT);
_delay_ms(2);
PORTB |= (1 << PAR_ACK_BIT);
}

```

The data byte is assembled as follows:

```
(PINC & 0x3f)
```

Reads the lower six bits of Port C, corresponding to Receiver A0–A5.

```
(PIND & 0xc0)
```

Reads the upper two bits of Port D, corresponding to Receiver D6 and D7.
These two values are combined to reconstruct the original eight-bit character.

Main Receiver Function

```

int main(void) {
    uart_init(9600);
    parport_init();

    while (true) {
        parport_loop();
    }

    return 0;
}

```

The Receiver initializes the UART and parallel interface, then continuously waits for and processes incoming bytes.

6. Design Notes and Verification Considerations

1. **Common ground is required.**
The Sender and Receiver must share a valid electrical ground reference, as specified by the GND net.
2. **The supplied wiring matches the primary protocol signals.**
The data, strobe, acknowledge, and busy mappings in the net list correspond to the firmware's intended operation.
3. **The Receiver code uses direct AVR register access.**
It is written for the ATmega328P-style hardware used by an Arduino UNO. It is not portable to arbitrary Arduino-compatible boards without modification.
4. **No UART output connection is documented in the net list.**
The Receiver transmits serial data internally through its UART, but the supplied circuit does not specify a connected serial terminal, USB interface, or external serial device.
5. **No timeout protection is implemented.**
Both the Sender and Receiver use blocking waits. A missing cable, incorrect signal level, or failed counterpart can stop normal execution indefinitely.
6. **The Receiver's Select signal is not wired.**
The firmware drives a Select-related output, but no corresponding Sender connection exists in the provided net list. It is therefore not part of the active communication path.